7-1 · LangChain 챗봇 기초 · 실습 1H

대화 이력을 가진 챗봇 실습

기억은 어디에서 오는가 — 실험으로 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 이력 없이 물으면 기억하지 못하는 것을 눈으로 본다

② 메시지 목록을 함께 보내 이름을 기억하는 챗봇을 만든다

③ 이력을 보내지 않는 판으로 되돌려 기억이 사라지는 것을 확인한다

④ 턴이 늘수록 입력 토큰이 커지는 것을 보고, 자르는 챗봇으로 개선한다

오늘 챗봇은 아직 우리 데이터를 모릅니다 — 문서를 붙이는 것이 이번 주 나머지 나흘의 일

실습 흐름

여섯 단계 — 기억을 만들고, 빼 보고, 잘라 낸다

앞 실습의 가상환경과 rag_env.py 를 그대로 씁니다 — 7단계 미니 과제는 시간이 남으면

1. 이력 없이	2. 이력 보내기	3. 콘솔 챗봇	4. 일부러 빼기	5·6. 토큰·자르기
nohistory.py 기억 없음	history.py 목록 append	chatbot.py while 루프	chatbot_nomem.py 기억 사라짐	tokens.py chatbot_trim.py

1·2단계

기억이 없는 상태 → 이력을 함께 보낸 상태

1단계 · nohistory.py

두 호출이 서로 독립 — 이름을 모른다



VS

2단계 · history.py

목록 전체를 다시 보내니 이름을 기억한다



3단계 — 콘솔 챗봇

chatbot.py — 이력을 쌓는 while 루프

1. **history = [SystemMessage(...)] · from rag_env import llm**
   친절한 데이터 엔지니어링 학습 조교 — 한국어로 짧게
2. **q = input("나: ") → history.append(HumanMessage(content=q))**
   quit · exit 를 치면 break
3. **answer = llm.invoke(history) → history.append(answer)**
   지금까지의 대화 전체를 보낸다
4. **입력 — 내 이름은 홍길동이야 → 내 이름이 뭐라고 했지? → quit**
   홍길동이라고 답하고 (이력 3개 메시지) → (이력 5개 메시지)(실측)

3 → 5

시스템 1 +

질문 · 답 두 쌍

4단계 — 일부러 실패시키기

chatbot_nomem.py — 이력을 보내지 않는 판

고쳤다는 판정은 되돌려서 확인합니다 — 3단계와 같은 세 줄을 입력해 보세요

바뀐 한 줄	보내는 것	실측	결론
invoke([system, HumanMessage(q)])	시스템 메시지 + 이번 질문만	「내가 알려준 이름은 없어요」	이력 전달이 곧 기억

5단계 — 길어지는 이력

tokens.py — 턴마다 보내는 토큰이 커진다

1. **for i in range(5): 질문 append → invoke(h) → 답 append**
   턴마다 a.usage_metadata['input_tokens'] 와 이력 개수를 찍는다

2. **입력 토큰 51 → 83 → 121 → 159 → 197**
   턴마다 약 40 씩 는다(실측 · Ollama)

3. **이력 3 → 11 개**
   이력 전체가 매번 가므로 비용과 응답 시간이 함께 는다

5턴 뒤

51 → 197

토큰

6단계 — 자르는 챗봇

chatbot_trim.py — 시스템 메시지는 지키고 최근 N개만

1. **MAX_MSGS = 4 — 시스템 메시지를 뺀 최근 메시지 수**
   질문 · 답 두 쌍

2. **trim — [history[0]] + history[-max_msgs:]**
   보낸 이력 1 → 3 → 4 → 4 · 입력 토큰 51 → 129 → 192 → 198(실측)

3. **입력 — 이름 → 수원에 살아 → 김치찌개 → 내 이름이 뭐라고 했지?**
   보낸 이력이 4 에서 멈췄는데도 넷째 질문에 「홍길동이네요!」(실측)

4. **첫 질문은 잘렸지만 답에 이름이 남았다**
   봇 답이 「수원에서 살고 계신 홍길동」 — 자르는 것은 메시지 개수, 정보가 아니다

실무에서는

대화를 요약해

시스템에 넣기

7단계 — 미니 과제

시간이 남으면 세 가지 중 하나

01

잊는 지점 찾기

MAX_MSGS 를 2 로, 시스템에 「이름을 다시 말하지 말라」

02

모르면 모른다고

규칙을 넣고 unknown.py 질문을 다시 물어 보세요

03

대화 저장·불러오기

chatbot.py 에 파일 저장과 다시 읽기를 더해 보세요

두 번째가 중요 — 「모르면 모른다」 지시만으로도 환각이 줄지만 없어지진 않아, 근거를 주는 RAG 가 필요하다

확인 포인트

이 1시간이 끝날 때 확인할 것

1. nohistory.py 의 둘째 답이 내 이름을 모른다
2. history.py 와 chatbot.py 는 이름을 기억해 답한다
3. chatbot.py 의 이력이 3개 → 5개 메시지로 는다
4. chatbot_nomem.py 는 이름을 모른다 — 이력 전달이 곧 기억
5. tokens.py 의 입력 토큰이 턴마다 약 40 씩 커진다
6. chatbot_trim.py 는 보낸 이력이 4 에서 멈춰도 답에 남은 이름으로 기억한다

7-1 · 실습 1H 마무리

다음은 정리·회고 (0.5H)

LangChain 부품 일곱 개를 한 장으로 정리하고, 7-2 문서 처리를 위한 패키지를 준비합니다

- ✓ **부품 일곱 개 — 역할과 코드 모양**
- ✓ **오늘의 체크리스트**
- ✓ **자주 겪는 오류 네 가지**
- ✓ **7-2 준비: 패키지 셋 설치**

가져갈 것

chatbot.py 와 5턴 토큰 수치